ArthLens

Build Handbook and Engineering Specification

An agentic, evidence-gated economic decision engine for Indian micro-enterprises

Version	1.0 (final handoff draft), 17 October 2026
Audience	A developer who is new to this project, and Antigravity (AI coding agent) working from this document
Project type	Major project: placement portfolio + IIITA OOP evaluation + possible startup foundation
Provisional name	ArthLens (name availability NOT verified)
How to read	Part 0 first. Then Parts 1-3 for understanding, Parts 4-12 for specification, Part 13 for the build order, Appendix A for ready-to-use agent prompts.

What this document is. It is a complete, self-contained plan that assumes the reader knows nothing about the project. It merges the research, the vision document and the MVP scope document into one set of decisions, and it corrects claims in the earlier drafts that were not supportable (Part 16).

Status tags used throughout. [DECISION] = settled, build it this way. **[VERIFY]** = a fact or limit that has NOT been confirmed; check it against the official source before relying on it, and record the result in docs/data-sources/ or docs/open-questions.md. **[ILLUSTRATIVE]** = an example number used only to explain a concept; it is not a fact and must never appear in the product as if it were one.

Part 0. Read this first

0.1 The project in five sentences

ArthLens helps a small entrepreneur decide whether and where to start a small business in an Indian district. It gathers public data, checks which government loan or subsidy schemes apply using exact rules (not AI guesses), models the money (costs, loan repayments, cash flow, risk), and compares the options. An AI agent plans and runs this investigation step by step, adapting when data is missing. The signature feature is the **Epistemic Ledger**: every number in the product is labelled as observed, an official rule, a user input, a calculation, an estimate or a hypothetical, and the system refuses to claim more than the evidence supports.

0.2 Non-negotiable rules for everyone building this (including Antigravity)

1. **Never invent facts.** No made-up datasets, scheme rules, statistics, URLs, prices, free-tier limits, benchmark results or resume numbers. If something is marked [VERIFY], check it; if you cannot, write it in docs/open-questions.md and continue with a clearly labelled placeholder.
2. **The LLM never calculates and never writes final numbers.** All maths, geospatial, financial and eligibility logic is deterministic Python. The LLM may interpret goals, propose plans from the registered tool list, extract draft rules from documents (a human approves them), and write prose around placeholders like {{q:UUID}}.
3. **Build to the interfaces in Part 5.** Do not change an interface signature without updating the UML (docs/uml) and the contract tests in the same commit.
4. **Follow the phase order in Part 13.** Do not start a phase until the previous phase's acceptance tests pass. The ledger (Phase 1) comes before data ingestion and before any agent code.
5. **Tests first for the ledger, finance and rule engine.** These are the correctness core. Write the property tests listed in Parts 4, 6 and 12 before the implementation.
6. **Every external dependency sits behind an adapter** (data provider, LLM, storage) with a local fixture or fake implementation so tests and demo mode never need the network.
7. **Label uncertainty everywhere.** Anything estimated, assumed or hypothetical must be visibly labelled in the API response and the UI, not only in documentation.
8. **Small commits, one task ID per commit** (task IDs are in Part 13). Keep a running docs/DECISIONS.md log for any deviation from this handbook, with the reason.
9. **Security is not polish.** Backend-enforced authorization, input validation and tool allowlists are built in the phase they first matter, not at the end.

0.3 Who does what

Role	Responsibility
Developer A (backend, algorithms, data)	Ledger, ingestion, geospatial and financial engines, database, tests for correctness, benchmark harness.
Developer B (agent, full stack)	Auth/RBAC, frontend (Atlas, Studio, Console, Policy), agent orchestrator, LLM adapter, exports, deployment.
Antigravity (AI coding agent)	Writes code and tests for tasks assigned from Part 13, using the prompts in Appendix A. Works inside the repository, one task at a time.
Human review (both developers)	Reviews every pull request against the acceptance criteria. Owns all [VERIFY] checks, data licensing, scheme-rule encoding sign-off, and the benchmark labels. The AI agent must not self-approve any of these.

0.4 Honest expectations about the schedule

An AI coding agent speeds up writing code. It does **not** speed up: verifying data sources and licences, reading scheme guidelines, authoring a benchmark, debugging free-tier deployment limits, or reviewing correctness. The realistic plan in Part 13 is about 16 weeks for two developers. If you have only about six weeks, build the **Minimum Demo Slice** described in Part 13.3 instead and say so openly; do not claim the full scope.

Part 1. What we are building and why

1.1 The problem

Someone with a small amount of capital (say a food-processing unit funded by a few lakh rupees) faces scattered public data, long policy PDFs, unclear local demand and loan arithmetic. Existing tools either only list schemes or sell location analytics to large companies. None that we found shows, for every number, whether it is observed, a rule, a calculation, an estimate or a guess.

1.2 The solution in plain language

- The user opens a map and picks a district, a business type and 2-3 candidate sites.
- An agent investigates: it fetches public data, notes what is missing, checks scheme eligibility with a rule engine, computes travel-time catchments and demand estimates, builds a cash-flow model, runs risk and sensitivity analysis and compares the sites.
- A verifier blocks any statement stronger than the evidence (for example, presenting a district average as a village measurement).
- The output is a report where every number can be clicked to see its sources, method and assumptions, and where the user can change assumptions and re-run.

1.3 What ArthLens deliberately does NOT do

- It does not forecast district GDP, village-level sales, or the causal effect of a policy.
- It does not simulate the whole Indian economy and does not use an LLM as an economic simulator.
- It does not guarantee business success or loan approval, auto-submit applications, or score individuals' creditworthiness.
- It does not scrape private platforms or invent local market data. Where data does not exist it says so.
- It gives decision support with stated assumptions. It is not legal or financial advice.

1.4 Our one technical contribution

The Epistemic Ledger with evidence gating. Each quantity records provenance, geographic and temporal support, uncertainty and derivation history. These propagate through calculations. A verifier checks every claim in the output, and the report renderer accepts numbers only from the ledger. We evaluate whether this reduces unsupported numeric claims compared with simpler pipelines.

Honest framing. Provenance tracking itself is not new (data lineage, W3C PROV and citation-grounded retrieval exist). Our claim is narrower: the specific combination of propagated evidence class and resolution, claim gating, and ledger-only number rendering, inside a stateful agent loop, measured against baselines. Whether this is novel is **[VERIFY]**: do a short literature check before using the word 'novel' anywhere.

1.5 Closest existing products (summary of research)

Product	What it does (from public descriptions)	Overlap and difference
Retail Vista (Jio)	Internal retail expansion tool: all-India H3 grid, 61 datasets, root orchestrator with four specialist sub-agents, backtested against store outcomes.	Closest overlap: agentic plus spatial plus backtest. Uses private data and is closed. We use only public data and focus on micro-enterprise decisions with evidence gating.
GeoIQ	Location intelligence with 100 m grids built from 600+ sources. Dealroom lists its status as closed [VERIFY].	B2B, proprietary prediction. No scheme or financing layer seen.
GapMaps Live	Cloud map platform for catchment metrics and location reports.	Enterprise retail focus; no scheme eligibility or per-number evidence labels seen.
PhonePe PulsePro (launched July 2026)	Hyperlocal business insights from aggregated transaction data down to postal-code level.	Private data moat we cannot match. Enterprise-oriented.
myScheme (NeGD)	Scheme discovery across 4000+ schemes with eligibility questions.	Discovery only. No cash-flow impact or business decision linkage.

Product	What it does (from public descriptions)	Overlap and difference
EconAgent (ACL 2024)	LLM agents simulating households and firms in a macro setting.	Research code. The LLM-as-simulator approach we deliberately avoid.
SHRUG (research platform)	Open dataset covering about 600,000 villages and towns.	A data source for us, not a competitor.

Not yet checked [VERIFY]: ONDC, Udyam, MSME Sampark, Zoho/Tally-style SME tools, general deep-research agents. Check these in Phase 0 before writing any 'gap' claim.

1.6 Glossary for newcomers

Term	Meaning
Agentic	An AI system that plans, picks tools, looks at results and decides the next step instead of following a fixed script.
Orchestrator	The program that runs the agent loop and enforces limits. Deterministic code, not the LLM.
Tool	A typed function the agent may call (for example calculate_loan_schedule). Has a schema, preconditions and tests.
Ledger / Quantity	The store of every number with its labels. A Quantity is one number plus its provenance, resolution, uncertainty and parents.
Provenance	Where a number came from: observed data, official rule, user input, calculation, estimate or scenario.
Resolution	How fine the geography is: nation, state, district, sub-district, village/urban area, point.
Gating	Blocking or downgrading a claim that is stronger or finer than its evidence.
Catchment	The area reachable within a given travel time from a site (computed with road-network shortest paths).
Huff model	A gravity model that splits demand between competing facilities by attractiveness and distance.
Monte Carlo	Running a model many times with randomly sampled uncertain inputs to see the spread of outcomes.
Sensitivity (tornado)	Changing one input at a time to see which assumptions move the result most.
Kleene 3-valued logic	Logic with True, False and Unknown. Used so a missing fact gives 'Unknown', not a guess.
AST	Abstract syntax tree. Scheme rules are stored as a tree of AND/OR/NOT/condition nodes.
LGD	Local Government Directory: official codes for states, districts, blocks, villages [VERIFY availability and format].
PostGIS	Geospatial extension for PostgreSQL.
PMTiles	A single-file vector map tile archive that the browser can read with HTTP range requests, so no tile server is needed.
SSE	Server-Sent Events: one-way live updates from server to browser, used for the agent console.
Replay mode	Showing a stored, clearly labelled recording of an earlier agent run when the live model is unavailable.
INR / lakh	Indian rupees; 1 lakh = 100,000.

Part 2. Scope: what is MVP and what is not

The earlier vision documents list 43 capabilities and 7 user roles. They are the long-term vision. Building all of them from scratch in one semester would produce a broad, shallow, untested product. The decision below is binding for the first release.

2.1 Scope tiers [DECISION]

Tier	Contents
MVP (build first)	One pilot district (default: Prayagraj, UP; confirm in Phase 0 against data availability). Two business templates: neighbourhood retail and a small food-processing/tiffin unit. Three schemes: PMFME, PMEGP, PM-MUDRA (confirm each is current in Phase 0). Roles with UI: Entrepreneur, Data Steward, Administrator. Workflows: A (verified answer), B (opportunity to decision), C (scheme to financial impact). Epistemic Ledger and verifier. Atlas, Scenario Lab, Agent Console, Policy Workspace, Evidence Graph. Report plus XLSX/PDF/JSON/CSV export. Replay mode. Public deployment.
First release (R1)	Second district ingested via configuration only. User data upload and Loop D (actual versus projected). LLM-assisted rule extraction with steward approval. Policy version diff. Sobol sensitivity. Hindi. DOCX/PPTX export. Community demand layer (aggregated, separate from observed data). Supplier allocation via min-cost flow. Customer and Investor roles.
Advanced roadmap	Site optimization at scale (MILP), policy change monitoring, supplier and programme-officer roles, portfolio analytics, relationship explorer, wider-effects simulation, intervention evaluation.
Rejected	Generic economics chatbot, concept explainer, household simulator, LLM-agent economy simulation, district GDP forecasting, individual credit scoring, auto-submission of applications, scraping private platforms, blockchain, vector database in MVP.

Roles and the database. The database role table and authorization layer are designed for all seven roles from day one (cheap), but only three roles get user interfaces in the MVP (expensive). A marketplace needs many real participants; do not build empty screens for them.

2.2 The 43 capabilities, tiered

#	Capability	Tier	Note
1	Natural-language economic research	MVP	Loop A
2	Economic indicator explorer	MVP	Choropleth plus charts
3	Economic concept explainer	Rejected	Generic LLM text, no computation
4	Economic relationship explorer	Advanced	Needs a curated knowledge base we do not have
5	Local economic intelligence (district profile)	MVP	Pilot district only
6	Economic event timeline	R1	
7	Comparative economic reports	R1	
8	Claim-by-claim fact-checking	MVP	Numeric claims first
9	Cross-source verification	MVP	Numeric contradiction detection
10	Government policy understanding	MVP	3 schemes
11	Policy version comparison	R1	AST diff
12	Policy impact navigation	MVP	Direct financial effects only
13	Eligibility engine	MVP	Kleene 3-valued
14	Policy change monitoring	Advanced	MVP: manual hash-check script
15	Interactive economic sandbox	MVP	Scenario Laboratory
16	Household financial simulator	Rejected	Different user and domain; scope creep
17	Business financial twin	MVP	
18	Price and break-even simulator	MVP	
19	Policy scenario simulator	MVP	Direct effects only; wider effects Advanced
20	Financing simulator	MVP	
21	Business feasibility simulator	MVP	
22	Sensitivity and risk analysis	MVP	OAT now; Sobol R1

#	Capability	Tier	Note
23	Monte Carlo simulation	MVP	
24	Alternative-strategy optimizer	MVP	Enumeration for small sets; MILP Advanced
25	Scenario comparison and replay	MVP	Bit-identical reruns
26	Personal/business profile	MVP	Business profile only
27	Micro-enterprise planning	R1	MVP provides the report
28	Actual-versus-projected tracking	R1	Needs user time series
29	Working-capital planning	MVP	
30	Market and competitor analysis	MVP	OSM completeness caveat shown
31	Goal-driven investigation planning	MVP	
32	Adaptive tool selection	MVP	
33	Iterative investigation and replanning	MVP	
34	Evidence-to-decision provenance graph	MVP	Evidence Graph screen
35	Persistent workflow and task management	MVP	Resume after interruption
36	Proactive monitoring	Advanced	
37	Structured report generator	MVP	
38	Custom-format information generator	MVP	Validated JSON export
39	Evidence-pack generator	R1	
40	Export studio	MVP	XLSX, PDF, JSON, CSV in MVP; DOCX, PPTX in R1
41	Multilingual assistant	R1	English first; Hindi with number-preservation check
42	Data upload and analysis	R1	Needs hardened parser
43	Portfolio analytics	Advanced	

2.3 The three MVP workflows

	Loop A: Question to verified answer	Loop B: Opportunity to decision	Loop C: Scheme to financial impact
Input	Economic question or claim	District, business type, 2-3 user-chosen sites, capital, assumptions	User business profile and facts, a scheme or 'what applies to me'
Agent does	Identify needed indicators; fetch; check unit/period/geography compatibility; compute; decompose claims; verify	Fetch data; compute catchments; allocate demand; build cash flow; Monte Carlo; sensitivity; compare sites	Evaluate eligibility (3-valued); ask for missing facts; compute subsidy and loan schedule; effect on cash flow
Output	Source-backed answer, or a clear 'cannot establish' section with how to resolve it	Ranked comparison with assumption labels and follow-up tasks	Eligible / Ineligible / Unknown with trace, and rupee impact
Fails when	Required data absent: report gap, never invent	Road graph or competitor data absent: fall back (flagged) or ask user	Rule version not valid on the date, or facts missing: Unknown plus questions
Acceptance	Every number traces to a ledger record; unanswerable question yields a gap report	Seeded rerun is bit-identical; zero constraint violations; spatial fallbacks labelled	Result equals hand-computed truth table; rupee impact equals golden calculation

Deferred loops: D (observed problem to intervention: needs user time series), E (event to scenario: needs calibrated data), F (multi-stakeholder coordination: needs trust and abuse rules).

The earlier example question asked the agent to *identify* candidate locations. For the MVP the user chooses 2-3 sites on the map (the app may list OSM-tagged industrial or commercial areas as *suggestions*, labelled as not checked for zoning or availability). Automatic candidate generation is an R1+ feature.

Part 3. System architecture, stack and repository layout

3.1 Architecture overview

```
Browser (Next.js: Atlas, Studio, Agent Console, Policy, Evidence Graph)
   |  REST (JSON) + SSE (live run events)
   v
FastAPI API  --  auth, RBAC, validation, rate limits
   |                          |
   | enqueue job              | read/write
   v                          v
jobs table (Postgres) <--- PostgreSQL + PostGIS  (ledger, runs, policies, geo, audit)
   ^
   | claim with FOR UPDATE SKIP LOCKED
Worker process (same codebase)
   |-- Agent orchestrator (state machine, budgets, checkpoints)
   |-- Tool registry -> engines: geospatial, demand, finance, MC, sensitivity, rules
   |-- Ledger + Verifier (gates every output)
   |-- ILLMClient (router + circuit breaker + replay/fake client)
   |-- DataProviders (data.gov.in, RBI, OSM, SHRUG, ...) behind adapters
   v
Object storage: PMTiles map archive, policy PDFs, report exports
```

3.2 Technology decisions [DECISION]

Layer	Choice	Reason / note
Frontend	Next.js (App Router), TypeScript, Tailwind	Standard, deployable on static/edge hosting.
Map	MapLibre GL JS (+ deck.gl optional for hex/heat layers)	Open source; reads PMTiles directly.
Graphs	Cytoscape.js for the Evidence Graph and the rule-tree view	One graph library instead of two.
Charts	ECharts (tornado, fan chart, histogram, comparison)	One chart library.
Backend	Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2, Alembic, GeoAlchemy2	Numeric and geospatial ecosystem.
Numerics	NumPy, SciPy, pandas, GeoPandas, Shapely, Pint (units), NetworkX (reference/baseline graph checks)	Dijkstra is implemented by us with heapq and checked against NetworkX in tests.
Optimization	Enumeration in MVP; SciPy milp or OR-Tools later	Do not use a solver where enumeration of 3-5 sites is exact.
Database	PostgreSQL + PostGIS only	No graph DB, no vector DB, no Redis in MVP. Add only if a measured need appears.
Jobs	Postgres table + worker using FOR UPDATE SKIP LOCKED	No Celery/Redis: fewer moving parts.
Live progress	SSE from run_events table, polling fallback	Works on common hosts.
Auth (MVP)	Email + password (argon2 or bcrypt), JWT in httpOnly secure cookie, seeded demo accounts per role	No external dependency during evaluation. Google OAuth in R1.
Agent engine	Custom state machine in Python behind an Orchestrator interface	State model and recovery logic must be visible and testable. LangGraph is a credible alternative (it offers checkpointing and interrupts) if the schedule slips; keep the same interface.
LLM access	ILLMClient interface with: provider adapters, router, circuit breaker, ReplayLLMClient, FakeLLMClient	Providers and free-tier limits change; see Part 8.6.
Tests	pytest, Hypothesis, Playwright, GitHub Actions	
Observability	Structured JSON logs with run_id, metrics table (step latency, tokens, cost)	Error tracking service optional.

[VERIFY] hosting-related claims (PostGIS availability on the chosen managed Postgres, object-storage free tier, free-hosting sleep behaviour, background-process support) are handled in Part 14.

3.3 Repository layout [DECISION]

```
arthlens/
  frontend/
    app/  (routes: atlas, studio, agent, policy, provenance, steward, admin, replay, reports)
    components/  maps/  charts/  agent-console/  api-client/
  backend/
    arthlens/
      domain/           Quantity, Scenario, BusinessProfile, AgentRun (state), PolicyVersion, enums
      ledger/           ledger.py (DAG), propagation.py, uncertainty.py, units.py,
                        specs/ (9 verification specifications), verifier.py, renderer.py (number-safe)
      policy/           rules/ (RuleNode, And, Or, Not, Leaf), visitors/ (evaluate, diff, impact),
                        kleene.py, versioning.py, schemes/ (JSON rule files)
      geo/              hierarchy.py, crosswalk.py, road_graph.py, dijkstra.py,
                        catchment.py, disaggregation.py
      demand/           allocator.py (interface), nearest.py, huff.py
      finance/          model_base.py (template), business_twin.py, loan.py, cashflow.py, breakeven.py
      simulation/       interfaces.py (SimulationModel), montecarlo.py, sensitivity.py,
                        scenarios.py, repro.py
      optimization/     solver.py (interface), enumeration.py, milp.py (later)
      agent/            orchestrator.py, run_state.py (State pattern), planner.py, goal.py,
                        tool_base.py, tools/ (one file per tool), decorators.py, budget.py,
                        recovery.py, memento.py, events.py
      llm/              client.py (ILLMClient), router.py, breaker.py, providers/, replay.py, fake.py,
                        firewall.py (untrusted-text wrapping)
      data/             pipeline.py (template method), providers/ (adapters), quality.py, staleness.py
      reports/          builder.py, sections.py, exporters/ (xlsx, pdf, json, csv; docx, pptx later)
      api/              routes/, schemas/, deps/ (auth, rbac), sse.py
      infra/            db/, repositories/, jobs/, storage/, observability/, config.py
    migrations/  (Alembic)
    tests/  unit/  property/  contract/  integration/  agent_eval/  e2e/  fixtures/
  benchmark/   tasks/ (dev, validation, frozen_test), labels/, harness/, results/
  deployment/  Dockerfile, compose.yaml, env.example, smoke/
  docs/        uml/ api/ data-sources/ architecture/ DECISIONS.md open-questions.md
  README.md
```

One home per concept. Quantity lives in domain/. Propagation and verification live in ledger/. Rule nodes live in policy/. The SimulationModel interface lives in simulation/. Do not duplicate.

Part 4. The Epistemic Ledger: specification

This is the heart of the project. Everything else (agent, reports, benchmark) depends on it. Implement and test it before ingestion and before any agent code. **These rules are decisions; if one proves unworkable, record the change in docs/DECISIONS.md and update the tests.**

4.1 The Quantity record (13 fields, immutable)

#	Field	Meaning
1	id (UUID)	Unique identifier.
2	definition_key	Semantic definition (for example population.total, scheme.pmfme.subsidy_rate).
3	value + unit	Number and a Pint unit (INR, INR/month, kg/month, persons, dimensionless).
4	kind	OBSERVED, POLICY_RULE, USER_ASSUMED, ESTIMATED, SCENARIO, CALCULATED, SIMULATED (COMMUNITY_REPORTED in R1).
5	evidence_floor	Weakest base class among all ancestors (see 4.2). Equals own base class for base quantities.
6	geo_unit_id + resolution	Geography the value actually represents and its level: NATION, STATE, DISTRICT, SUBDISTRICT, VILLAGE_URBAN, POINT.
7	temporal support	as_of date, period start/end, and for rules: effective_from/effective_to.
8	uncertainty	EXACT, INTERVAL(lo, hi), DISTRIBUTION(name, params), SAMPLES(reference), or UNQUANTIFIED.
9	source_refs	List of (source_version_id, locator such as page, table, cell, or URL + retrieval date).
10	method_id + model_version	How it was produced; version of the code/model; code git hash for runs.
11	parent_ids (UUID[])	Inputs of a derived quantity. Must form a DAG; cycles are rejected.
12	quality_flags	STALE, INCOMPLETE_SOURCE, LOW_COVERAGE, CONTRADICTED, ROUNDED, and so on.
13	license_meta	Licence/attribution string of the source, shown in exports.

4.2 Provenance combination rule (evidence floor)

Base classes ranked by strength (higher is stronger): **OBSERVED = POLICY_RULE (4) > USER_ASSUMED (3) > ESTIMATED (2) > SCENARIO (1)**.

- A derived quantity (kind CALCULATED or SIMULATED) has **evidence_floor = minimum evidence_floor of its parents**.
- A CALCULATED value can never have kind OBSERVED, however exact the arithmetic.
- How a number may be *described* in a report: 'observed' only if kind is OBSERVED; 'calculated from official data' only if kind is CALCULATED and floor is 4; 'depends on a user input' if floor is 3; 'depends on an estimate' if floor is 2; 'hypothetical scenario' if floor is 1 or kind is SCENARIO.
- SIMULATED outputs inherit the floor of their inputs and carry the scenario id, seed, model version and data versions.

4.3 Resolution rule

- Order from coarse to fine: NATION(0) < STATE(1) < DISTRICT(2) < SUBDISTRICT(3) < VILLAGE_URBAN(4) < POINT(5).
- A derived quantity's resolution is the **coarsest** (lowest index) among its parents.
- **Exception:** a registered downscaling method (for example population-weighted allocation of a district total to zones) may produce a finer resolution. Its output has kind ESTIMATED, a floor of at most 2, the method_id, and must satisfy a conservation test (zone values sum back to the district total).
- A claim may not assert a resolution finer than the quantity's resolution (Specification V2).

4.4 Uncertainty combination

- EXACT with EXACT stays EXACT. Interval inputs use interval arithmetic (division only when the divisor interval excludes zero).
- **UNQUANTIFIED is contagious:** if any parent is UNQUANTIFIED, a deterministic child is UNQUANTIFIED.
- A Monte Carlo run is the only operation that turns input DISTRIBUTION/INTERVAL assumptions into SAMPLES. Its output carries the input distributions as parents.
- Staleness: each source has a configured maximum age. Beyond it the quantity gets the STALE flag; derived quantities inherit STALE.

4.5 The nine verification specifications

ID	Class name	Rule
V1	DerivationDowngradeSpec	A derived value is never labelled OBSERVED; the described class must not be stronger than the computed floor.
V2	GeographicPrecisionSpec	A claim's geography must not be finer than the evidence resolution unless a registered downscaling method applies.
V3	PolicyCitationSpec	A policy claim must cite a POLICY_RULE record whose version was valid on the stated date.
V4	SourceAttributionSpec	Every number in the output is a ledger quantity with at least one source reference or parent chain ending in sources. Numbers found in a document passage must first be ingested as ledger records with a passage locator.
V5	ContradictionVisibilitySpec	If two sources disagree beyond tolerance on the same definition, geography and period, both stay visible and the claim is marked disputed until an explicit resolution method is applied.
V6	UnitCompatibilitySpec	Operations require compatible Pint units.
V7	ReproducibilitySpec	Calculated and simulated quantities store method, versions, seed and parent ids sufficient to recompute.
V8	MissingAssumptionSpec	No default or invented assumption: a missing required input is either asked of the user or filled by a registered, labelled prior (kind SCENARIO).
V9	SimulationModelVersionSpec	Every simulation result names scenario id, model version, seed, N and data versions.

Specifications are independent classes combined with an AndSpecification inside LedgerVerifier. Each returns a result object with pass/fail, the offending quantity id, and a human-readable reason (shown in the Agent Console).

4.6 Numbers in reports: ledger-only rendering (rule G4)

- The LLM writes prose containing placeholders such as {{q:3f2a...}}. It must not type digits for results.
- The ReportRenderer substitutes each placeholder with the formatted value, unit and a clickable chip (class, resolution, as_of, link to the Evidence Graph).
- After rendering, a scanner rejects any numeric literal not on an allowlist (section numbers, years that appear in a ledger quantity's period, list indices). A rejected report goes back to the agent as a verification failure.
- Text in a quoted source passage is shown as a quote with its locator; numbers in it are not treated as results.

4.7 Required tests for the ledger (write first)

- Property: class never strengthens through derivation (floor of child equals min of parents).
- Property: resolution never refines without a registered downscaling method; downscaling conserves totals.
- Property: the graph rejects cycles; ancestors of a quantity are exactly its transitive parents.
- Property: UNQUANTIFIED parent gives UNQUANTIFIED child; interval arithmetic contains the exact result for sampled points.
- Unit tests: invalid unit combinations rejected; STALE flag appears and propagates.

- Policy: a rule not valid on the as-of date is rejected by V3.
- Report: an LLM output containing a free number is rejected; placeholders resolve correctly.
- Reproducibility: identical scenario, seed, model version and data versions give identical outputs (bitwise for floats under the same library versions).

Part 5. Object-oriented design: interfaces, patterns, SOLID

OOP is evaluated on real code and tests. Patterns are used only where they solve a design problem here; Singleton and similar are deliberately not used. The skeletons below fix the **interfaces**; bodies are implemented in the phases of Part 13.

5.1 Core interface skeletons (Python)

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from enum import Enum

class Kind(Enum):
    OBSERVED="OBSERVED"; POLICY_RULE="POLICY_RULE"; USER_ASSUMED="USER_ASSUMED"
    ESTIMATED="ESTIMATED"; SCENARIO="SCENARIO"; CALCULATED="CALCULATED"; SIMULATED="SIMULATED"

@dataclass(frozen=True)            # encapsulation: immutable once registered
class Quantity:
    id: str; definition_key: str; value: float; unit: str; kind: Kind
    evidence_floor: int; geo_unit_id: str; resolution: int
    as_of: str; uncertainty: "Uncertainty"; source_refs: tuple
    method_id: str; model_version: str; parent_ids: tuple
    quality_flags: frozenset; license_meta: str

class DataProvider(ABC):           # Adapter target
    @abstractmethod
    def fetch(self, query: "ProviderQuery") -> "Dataset": ...

class DemandAllocator(ABC):        # Strategy
    @abstractmethod
    def allocate(self, zones, facilities, params) -> "Allocation": ...
# Implementations: NearestFacilityAllocator, HuffAllocator

class SimulationModel(ABC):        # Strategy; Liskov contract tested for every implementation
    @abstractmethod
    def run(self, scenario: "Scenario", seed: int) -> "RunResult": ...

class RuleNode(ABC):               # Composite
    @abstractmethod
    def accept(self, visitor: "RuleVisitor"): ...
# Implementations: AndNode, OrNode, NotNode, LeafPredicate

class RuleVisitor(ABC):            # Visitor
    pass  # EligibilityEvaluator (Kleene), PolicyDiff, FinancialImpactExtractor

class VerificationSpecification(ABC):   # Specification
    @abstractmethod
    def check(self, claim, ledger) -> "SpecResult": ...

class Tool(ABC):                   # Template Method + Command
    def execute(self, raw_input):
        inp = self.validate_input(raw_input)
        self.check_preconditions(inp)
        out = self.run_core(inp)
        self.verify_postconditions(out)    # registers outputs in the ledger
        return out
    # validate_input, check_preconditions, run_core, verify_postconditions are abstract

class ILLMClient(ABC):
    @abstractmethod
    def generate_structured(self, prompt, schema, task_type) -> "BaseModel": ...

class RunState(ABC):               # State
    @abstractmethod
    def handle(self, run: "AgentRun", event) -> "RunState": ...

class Repository(ABC):             # Repository (one per aggregate)
    pass
```

5.2 Pattern map (each has a concrete test)

Pattern	Problem solved here	Where	Test that demonstrates it
Strategy	Swap algorithms without touching callers	DemandAllocator, SimulationModel, OptimizationSolver	One contract test suite run against every implementation
State	Run lifecycle with guarded transitions	AgentRun and RunState subclasses	Illegal transitions raise domain errors; COMPLETED cannot return to PLANNING
Command + Template Method	Uniform, auditable tool execution	Tool base class and 13 tools	Subclass cannot skip postconditions; invocation record can be replayed
Decorator	Cross-cutting concerns	Budget, Timeout/Retry, Provenance registration, Audit logging; Circuit breaker around LLM client	Wrap a flaky fake tool and observe retries and budget counts
Adapter	Heterogeneous external sources	DataProvider implementations	Swap in a fixture provider with no code change
Composite + Visitor	Policy rules as trees with several operations	RuleNode tree; evaluator, diff, impact visitors	Nested rules evaluate correctly; new visitor added without editing nodes
Specification	Composable verification checks	9 verification specs	Compose and report per-spec results
Observer	Real events to the UI	RunEventBus to run_events to SSE	Subscriber receives ordered events
Memento	Pause and resume without holding a thread	ExecutionMemento serialized per step	Kill worker mid-run; resume from last checkpoint
Template Method	Same ingestion skeleton for each source	DatasetIngestionPipeline	Two providers produce identical pipeline stage order
Repository	Persistence abstraction	*Repository classes	In-memory fakes in unit tests
Factory	Build tools/providers/models from config	ToolFactory, ProviderFactory	Config-driven construction
Builder (R1)	Format-agnostic report tree to several exporters	ReportDocumentBuilder	Same tree exports to XLSX and PDF

5.3 SOLID traceability

Principle	Where visible	Proof
Single responsibility	Policy engine, finance engine, ledger/verifier, orchestrator are separate packages	Each package tested without importing the others' internals
Open/closed	New allocator, provider or visitor added by a new class	A test registers a stub implementation with no edits to callers
Liskov substitution	Every SimulationModel and DemandAllocator honours the same contract (conservation, determinism, output types)	Shared contract tests parameterized over implementations
Interface segregation	Narrow interfaces: DataProvider.fetch, VerificationSpecification.check, Tool phases	No class implements methods it does not use
Dependency inversion	Orchestrator depends on Tool, ILLMClient and Repository abstractions	Whole agent run executes with FakeLLMClient and in-memory repositories

5.4 Composition vs inheritance

Inheritance only for true is-a families (RuleNode, Tool, RunState, allocators). Composition elsewhere: AgentRun has a Plan, a Budget and a current RunState; Scenario aggregates parameters, input quantities and constraints; decorators wrap tools instead of subclassing them.

5.5 UML deliverables (must match the code)

- Use-case diagram (Entrepreneur, Data Steward, Administrator, and the future roles shown greyed).
- Class diagram: domain, ledger, policy, agent, demand/simulation.
- Sequence diagrams: Loop A, Loop B, Loop C (including approval and resume).
- Activity diagram: policy evaluation with missing-information handling.
- State-machine diagram: agent run lifecycle (Part 8.2).
- Component and deployment diagrams (Part 3 and Part 14).
- Write them in Mermaid or PlantUML under docs/uml; regenerate when interfaces change.

Part 6. Algorithms and mathematical models

Each algorithm below has a purpose, inputs and outputs, assumptions, complexity, a simpler baseline to compare against, and a test strategy. Do not present modelled outputs as observations; they enter the ledger as CALCULATED, ESTIMATED or SIMULATED.

6.1 Travel-time catchments (Dijkstra)

- **Purpose:** which zones can reach a site within t minutes (use 15, 30, 60 as defaults).
- **Inputs:** road graph extracted around the site (vertices V, directed edges E with length and speed assumption), site point, t. **Output:** reachable vertex set with travel times; catchment polygon; zones intersected.
- **Algorithm:** Dijkstra with a binary heap (heapq), complexity O((V+E) log V). Implement it ourselves, and test against NetworkX on random graphs.
- **Assumptions:** OSM road completeness; speeds from road class defaults (labelled SCENARIO); no traffic or seasonal effects.
- **Baseline:** Euclidean buffer of equal radius. **Validation:** compare travel times with an independent router on sampled pairs; disconnected graph and missing-road tests; the site snapped to nearest road node with the snap distance reported.
- **Memory note:** never load the national road network. Query a bounding box around each site (start with about 15 km, configurable) through PostGIS spatial indexes.

6.2 Spatial downscaling of demand (registered estimation method)

Consumption data is typically available at coarse geography [VERIFY the exact survey and level]. To place demand in zones, use a registered method *popweight_v1*:

$E_z = D_{district} \times (pop_z \times w_z) / \text{sum over zones } (pop_k \times w_k)$

where D is the district-level demand total (a ledger quantity), pop is zone population, and w is an optional accessibility weight. The output is ESTIMATED at the zone resolution with method_id popweight_v1. **Conservation test:** the sum of E over zones equals D. **Baseline:** pure population share (w = 1). **Limit:** says nothing about real local spending differences.

6.3 Huff gravity demand allocation

For demand zone i, facility j with attractiveness A_j, travel time d_{ij} and decay exponent lambda:

$P(i \text{ to } j) = A_j \times d_{ij}^{-lambda} / \text{sum over k of } (A_k \times d_{ik}^{-lambda})$

$D_j = \text{sum over i of } (E_i \times P(i \text{ to } j))$

- **Inputs:** zone demand E_i (ESTIMATED), facilities (existing from OSM plus the user's candidate sites), A_j, d_{ij} (from 6.1), lambda. **Output:** allocation matrix and D_j per facility.
- **Calibration:** A_j and lambda cannot be estimated from public data. They are SCENARIO inputs with a documented prior range [VERIFY a literature range for lambda]. Sensitivity analysis on lambda is mandatory in every Loop B result.
- **Complexity:** O(|I| x |J|). Vectorize with NumPy.
- **Baseline:** nearest-facility allocation (each zone sends all demand to its nearest facility).
- **Tests:** row probabilities sum to 1; demand is conserved when there is no outside option; with lambda very large Huff approaches nearest-facility; both allocators pass the same contract tests.

6.4 Financial model (monthly, deterministic)

```
Revenue_t   = Quantity_t * Price
Quantity_t  = Q0 * (1 + g)^t * seasonality[month(t)]
COGS_t      = Quantity_t * VariableCostPerUnit
CashFlow_t  = Revenue_t - COGS_t - Opex_t - DebtService_t - Tax_t - OtherPayments_t
Cash_t      = Cash_(t-1) + CashFlow_t            (Cash_0 = equity + disbursed loan - capex)
Contribution = Price - VariableCostPerUnit       (must be > 0 or the model reports 'no break-even')
BreakEvenQ  = FixedCosts / Contribution
CCC (days)  = DaysInventory + DaysReceivable - DaysPayable      (working-capital need)
```

- Template Method skeleton in AbstractFinancialModel: initialize capital, compute revenue, deduct costs, apply debt and policy schedule, compute tax and working capital.
- Tax, inventory movements, owner withdrawals and timing differences are parameters, defaulting to zero only when the user explicitly confirms (V8).
- **Tests:** golden tests against a hand-built spreadsheet for each template; zero-growth, zero-interest, invalid-term and infeasible cases; cash never silently negative (flagged).
- **Baseline:** single-case deterministic projection.

6.5 Loans, moratorium and subsidy

Equal monthly instalment (EMI) with monthly rate r = annual_rate / 12 and n instalments:

$EMI = P \times r \times (1+r)^n / ((1+r)^n - 1)$ (if r = 0: EMI = P / n)

- Schedule rows: opening balance, interest, principal, closing balance, per month. The final balance must reach zero within rounding tolerance.
- **Moratorium and subsidy treatment are parameters, not assumptions:** interest during moratorium is PAID or CAPITALISED; credit-linked subsidy is modelled as ADJUSTS_PRINCIPAL, BACK_ENDED_RELEASE or LOCKED_DEPOSIT [VERIFY the actual mechanism for each scheme in its guidelines and encode exactly that].
- **Tests:** compare with an independent spreadsheet; property test that EMI is increasing in rate and in principal; total repaid >= principal.

6.6 Monte Carlo risk simulation

- **Purpose:** the spread of outcomes under uncertain inputs (demand capture rate, raw-material price, payment delay).
- **Method:** NumPy Generator(PCG64(seed)); sample PERT or triangular distributions from user-supplied or labelled-prior bounds; evaluate all trajectories as a (N x T) array. Default N = 2,000 for interactive use and 10,000 for reports (tunable).
- **Outputs:** percentiles of profit (P10/P50/P90), distribution of minimum cash balance, probability of cash shortfall, payback quantiles. Output kind SIMULATED with SAMPLES uncertainty and the input distributions as parents.
- **Convergence:** report the standard error of the shortfall probability; rerun with larger N if it exceeds a configured threshold.
- **Limit:** results are only as good as the input distributions. They are labelled USER_ASSUMED or SCENARIO accordingly.
- **Do not claim run-time figures** (for example 'under 30 ms') in any document until measured on the deployed host.

6.7 Sensitivity analysis

- **MVP:** one-at-a-time perturbation of each uncertain input across its range, producing a tornado chart. Cost: one model run per input per side.
- **R1:** Sobol variance-based indices (Saltelli sampling, N(2k+2) evaluations for k inputs). Test on functions with known analytic indices.
- **Agent use:** if one input dominates the outcome range, the agent adds a step asking the user for evidence on that input and creates a follow-up task.

- Any statement like 'two inputs account for X percent of variance' must come from a computed ledger quantity, never typed text.

6.8 Policy rule evaluation (Kleene three-valued logic)

A	B	A AND B	A OR B	NOT A
T	T	T	T	F
T	U	U	T	F
F	U	F	U	T
U	U	U	U	U
T	F	F	T	F

- Leaf predicates compare a user fact to a threshold or set. A missing fact yields U and is recorded as a question the agent can ask.
- The evaluator returns status (ELIGIBLE, INELIGIBLE, UNKNOWN), the list of unknown facts, and a trace of every node result. The LLM never decides eligibility.
- Rules are versioned (effective_from, effective_to). Evaluation takes an as-of date and refuses a version not valid on that date.
- **Tests:** exhaustive truth-table tests; nested trees; version selection by date; trace equals expected for fixtures.
- **Baseline:** flat questionnaire (as discovery portals use).

6.9 Other algorithms

Algorithm	Where	Notes
Numeric contradiction detection	Cross-source verification (V5)	Two values conflict if their tolerance intervals do not overlap for the same definition, geography and period. Test with injected conflicts.
Site comparison by enumeration	Loop B with 2-5 sites	Exact; no solver needed. MILP only in Advanced.
Min-cost flow (R1)	Supplier-to-buyer allocation	minimize sum c_ij x_ij subject to capacity and demand constraints; baseline greedy nearest; test constraints and cost not worse than greedy.
Full-text retrieval	Policy document search	PostgreSQL full-text search with section metadata. A vector index is not justified for a handful of scheme documents; add pgvector only if a retrieval benchmark shows benefit.

Part 7. Real-world data plan

The data layer decides what we may claim. Phase 0 exists to verify it. Nothing here is a guarantee that a dataset has the fields or granularity we want.

7.1 Source register (status as of this document)

Source	What research confirmed	Needs verification before use	Planned use
data.gov.in (OGD) API	An API key is needed; resource endpoints return JSON/CSV by resource id. Many resource ids exist.	Each specific resource id, fields, freshness, licence. Rate limits [VERIFY].	Indicators, programme data
RBI DBIE	Quarterly deposit and credit statistics classified by state, district, centre, population group and bank group; BSR returns include credit by occupation and district.	Exact tables, download method, licence terms on DBIE. A third-party mirror showed district deposits last updated 2022, so staleness risk.	Credit context
SHRUG (Development Data Lab)	Open platform linking about 600,000 villages and towns; includes census and economic-census derived data.	Licence for any non-academic use. Underlying data is 2011 census and 2013 economic census: STALE flag.	Population and non-farm establishments in the pilot district
OpenStreetMap	Not checked in research.	ODbL attribution and share-alike; POI and road completeness for the pilot district.	Roads, POIs, competitor counts (with completeness caveat)
Census 2011 / LGD codes	Not checked.	LGD availability and code format; crosswalk to census codes.	Geographic hierarchy, identifiers
NSS household consumption survey	Not checked.	Which round, which geographic level (likely state or coarser), food-category detail.	District/state per-capita spend (ESTIMATED downscaling)
Udyam registration statistics	Not checked.	Whether district-level counts are publicly downloadable.	Competitor/MSME counts if available
Scheme guidelines (PMFME, PMEGP, PM-MUDRA)	Scheme names only; myScheme lists thousands of schemes with eligibility questions.	Fetch official PDFs; record URL, date, SHA-256; read parameters directly. Do not copy numbers from any summary, including this handbook.	Rule trees and financial impact
Administrative boundaries	Not checked.	Official-boundary requirements for maps of India and licence of the chosen source.	Atlas, PMTiles

7.2 Phase 0 verification checklist (per source)

Create docs/data-sources/<source-id>.md with:

- Official URL and API documentation link; access method; whether a key or login is required.
- A live request or file inspection performed by a human on a recorded date; row count; list of fields; sample rows.
- Geographic granularity and the identifier scheme; temporal coverage; update frequency; the as_of date.
- Licence text copied verbatim with link; attribution requirement; commercial-use constraints.
- Known quality problems (missingness, duplicates, unit issues).
- Fallback if the source is unavailable (cached snapshot in object storage).
- A GO / NO-GO line with reason. A NO-GO source is replaced or its dependent feature is cut.

7.3 Go/no-go gate for the pilot district

- Usable road network and POIs for the district (checked by drawing the road graph and counting POIs by category).
- Population at the resolution the demand model needs (sub-district or village-level) with an as_of date.
- At least three schemes whose current official guidelines are fetched and readable.
- If any of these fails: change district or cut the dependent feature. Do not weaken the claim.

7.4 Resolution policy

- State and district analysis is expected to be supported. Village-level population and non-farm establishments may be available via SHRUG but are stale.
- No verified public source of village-level sales, prices or consumption was found. Any figure at that level is ESTIMATED (method registered) or refused.
- The Atlas shows two tiers: **navigable** (boundaries and identifiers exist) and **analysable** (the evidence needed for a supported workflow exists). The pan-India claim applies to navigability and architecture; analysis is limited to analysable areas.
- Acceptance test for the pan-India architecture: ingesting a second district requires configuration and a provider entry, not code changes in engines.

7.5 Ingestion pipeline (template method, 10 steps)

1. Retrieve the source.
2. Record retrieval date and version (hash).
3. Validate file or response schema.
4. Normalize field names and units (Pint).
5. Reconcile geographic identifiers (crosswalk to LGD).
6. Check missingness, duplication, outliers.
7. Record geographic and temporal support.
8. Record licence and provenance metadata.
9. Store normalized rows plus ledger quantities of kind OBSERVED.
10. Publish coverage and quality status (CURRENT, INCOMPLETE, STALE, ESTIMATED, UNAVAILABLE) for the Atlas data-quality layer.

Part 8. The agentic engine

Principle: the LLM proposes; deterministic software validates and executes; the verifier decides what may be claimed. Agentic behaviour means the next step depends on what has been found, not on a fixed sequence.

8.1 Components

Component	Responsibility	Implementation notes
Goal interpreter	Natural language to StructuredObjective (sector, region, capital, constraints, required outputs, known facts, missing inputs)	LLM with a Pydantic schema; invalid output retried once then the run asks the user.
Planner	Propose steps from the registered tool list only	LLM proposal; PlanValidator checks tool names, input schemas, preconditions and dependencies.
Tool registry	Typed tools with allowlist per run	Tool.execute template method; one file per tool.
Next-step policy	Choose next action from state gaps	Deterministic priority rules (8.4); LLM may suggest, validator decides.
Evidence manager	Registers every tool output in the ledger	ProvenanceRegistrationDecorator.
Verifier	Run the nine specifications and acceptance conditions	Gates COMPLETED.
Recovery engine	Handle failure, missing data, contradiction, invalid output	Rule table (8.4).
Approval gate	Pause for human approval	AWAITING_APPROVAL; resumes from memento.
Budget governor	Enforce limits	Counters checked at every transition.
Event bus	Append to run_events	Feeds SSE and replay.

8.2 Run state machine [DECISION]

States: CREATED, INTERPRETING, PLANNING, EXECUTING, OBSERVING, RECOVERING, AWAITING_USER, AWAITING_APPROVAL, VERIFYING, COMPLETED, FAILED, BUDGET_EXHAUSTED, CANCELLED. Terminal states: COMPLETED, FAILED, BUDGET_EXHAUSTED, CANCELLED. Illegal transitions raise a domain exception.

From	Event	To
CREATED	run accepted	INTERPRETING
INTERPRETING	objective valid	PLANNING
INTERPRETING	required slots missing	AWAITING_USER
PLANNING	plan validated	EXECUTING
PLANNING	plan invalid (bounded retries)	PLANNING or FAILED
EXECUTING	tool returned	OBSERVING
OBSERVING	next step ready	EXECUTING
OBSERVING	error, gap or contradiction	RECOVERING
OBSERVING	gated action reached	AWAITING_APPROVAL
OBSERVING	plan complete	VERIFYING
RECOVERING	retry or alternate tool	EXECUTING
RECOVERING	needs replan	PLANNING
RECOVERING	needs user fact	AWAITING_USER
RECOVERING	unrecoverable	FAILED
AWAITING_USER	answer received	PLANNING
AWAITING_APPROVAL	approved / rejected	EXECUTING / CANCELLED
VERIFYING	all checks pass	COMPLETED
VERIFYING	a check fails	RECOVERING
any non-terminal	budget limit hit	BUDGET_EXHAUSTED (partial result returned)

8.3 Budgets (initial defaults, tune after measuring)

Limit	Default	Behaviour at limit
Steps per run	25	BUDGET_EXHAUSTED with partial result and unresolved gaps listed
Replans per run	3	Same
Seconds per tool call	60	Timeout; retry per policy; then RECOVERING
Model calls per run	30	BUDGET_EXHAUSTED
Retries per transient tool error	2	Then alternate tool or RECOVERING
Cost per run	MAX_COST_USD_PER_RUN env, start at 0.25	BUDGET_EXHAUSTED. Measure real cost in Phase 6 and set the value from data.
Daily runs per user	Configurable quota	HTTP 429 with explanation; replay mode remains available

These numbers are starting hypotheses, not measured values. Enforce them in code (BudgetEnforcementDecorator), not in prompts.

8.4 Conditional execution: decision table

Observation	Action
Required slot missing (for example capital, site, product)	AWAITING_USER with a targeted question; persist memento; release the worker.
Data absent at required resolution (for example consumption at sub-district level)	Option 1: registered downscaling (popweight_v1), outputs ESTIMATED. Option 2: ask the user for local figures. Option 3: reduce scope. Option 4: report 'cannot establish' with resolution steps. Choose by rule order and what the user allowed.
Eligibility returns UNKNOWN	Ask only for the unknown facts listed by the evaluator; re-evaluate.
Two sources contradict beyond tolerance	Keep both visible; mark claim disputed; do not average silently.
Monte Carlo shows high shortfall probability	Try alternative financing structures (tool calculate_loan_schedule with other terms) within user constraints; re-run; report all tried.
Sensitivity shows one input dominates	Add a step asking for evidence on that input; create a follow-up task.
Verifier fails (for example free number, resolution overclaim)	RECOVERING: regenerate narrative or recompute; bounded by budget.
Tool times out	Retry with backoff; alternate provider or cached snapshot; flag in output.
Gated action	AWAITING_APPROVAL (publishing to shared view, steward rule activation).

8.5 The registered tools

Tool	Input -> output	Side effects	Approval
fetch_economic_indicator	{source, geo_id, series} -> Quantity[]	Ledger writes	No
search_policy_documents	{scheme, query} -> passages with locators	None	No
verify_claim	{claim} -> verdict + evidence	Ledger read	No
evaluate_scheme_eligibility	{scheme, facts, as_of} -> status, unknown facts, trace	None	No
calculate_loan_schedule	{principal, rate, term, moratorium, subsidy params} -> schedule Quantity[]	Ledger writes	No
calculate_break_even	{price, variable cost, fixed costs} -> Quantity	Ledger writes	No
compute_catchment	{site, minutes, mode} -> polygon, zones, travel times	Ledger writes	No
allocate_spatial_demand	{zones, facilities, allocator, params, seed} -> allocations	Ledger writes	No
run_business_scenario	{scenario} -> cash-flow Quantity[]	Run artifact	No
run_sensitivity_analysis	{scenario, inputs} -> ranked influence, tornado data	Run artifact	No
compare_scenarios	{run_ids} -> comparison with reasons	None	No
generate_report	{verified claim graph} -> report document	Report artifact	No
create_followup_tasks	{unresolved gaps} -> task objects	Persisted tasks	No
publish_to_shared_view	{run_id} -> shared record	Shared write	Yes

Every tool declares: purpose, input schema, output schema, preconditions, side effects, error conditions, retry policy, verification rules, and approval requirement. Decorators applied in order: BudgetEnforcement, TimeoutAndRetry, ProvenanceRegistration, AuditLogging.

8.6 LLM access layer

All LLM calls go through ILLMClient.generate_structured(prompt, schema, task_type). No module imports a provider SDK directly. A router chooses the provider by task type; a circuit breaker fails over on rate-limit or error; ReplayLLMClient serves recorded responses for demo and tests; FakeLLMClient returns scripted output for unit tests.

Task type	Needs	Suggested approach
Goal interpretation, tool routing	Fast, reliable structured JSON	A fast model on a provider with a free or cheap tier [VERIFY current models, limits and terms at build time]
Replanning reasoning	Multi-step reasoning	A stronger reasoning-capable model; keep calls few
Policy PDF to draft rule tree (R1)	Long context, citations	A long-context model; every cited quote must exist verbatim in the PDF; a steward approves
Report narrative and Hindi (R1)	Fluent text that keeps placeholders	Any capable model; the number-preservation check enforces correctness

[VERIFY] The earlier draft listed specific model names and free-tier numbers (requests per day, tokens per minute) and a model name that may not exist. Provider terms, model names and free limits change often. Do not hard-code them; keep them in config, confirm at build time, and design assuming limits may be hit (circuit breaker plus replay mode). Do not promise unlimited free live AI.

8.7 Prompt-injection firewall

- Retrieved passages, uploaded documents and community text are wrapped in clearly delimited untrusted-data blocks and passed in a separate message role from instructions.
- The model's output is parsed into typed objects. A plan can only reference registered tools; free text cannot call tools or change budgets.
- Tool allowlists are per run. Tools have no network access except through allowlisted fetchers (domain list, no private IP ranges, timeouts, size limits).
- Tests: documents containing instructions ('ignore previous rules', 'call publish_to_shared_view') must produce zero tool authorizations.

8.8 Worked example: Loop B (numbers are [ILLUSTRATIVE])

1. Entrepreneur selects the pilot district, sector food processing, capital 5 lakh INR, three candidate sites, and asks for a comparison.
2. Run created (CREATED), budgets set. Goal interpreter returns StructuredObjective; plan validator accepts a 9-step plan.
3. fetch_economic_indicator pulls population, establishments and banking indicators (OBSERVED, with as_of and STALE flags where applicable).
4. Data-gap detection: consumption exists only at district level, so the orchestrator enters RECOVERING and applies popweight_v1, producing ESTIMATED zone demand.
5. compute_catchment runs Dijkstra for each site; allocate_spatial_demand runs Nearest and Huff and stores both.
6. evaluate_scheme_eligibility returns ELIGIBLE / UNKNOWN per scheme; the agent asks for unknown facts if needed; calculate_loan_schedule uses the scheme parameters from the approved rule version.
7. run_business_scenario, Monte Carlo and OAT sensitivity run; if shortfall probability is high the agent tries alternative financing and re-runs.
8. compare_scenarios ranks sites; VERIFYING runs V1-V9; generate_report renders ledger-only numbers; create_followup_tasks lists evidence to gather (for example local price quotes for the dominant input).

Part 9. Database design

PostgreSQL with PostGIS. Alembic migrations. Candidate schema below; the exact column set may evolve but the ownership and access distinctions must remain: **public** data, **user-owned** data, **shared** (approved) data, and **derived run** records.

9.1 Core tables

Table	Key columns / purpose
users, roles, user_roles	email, password_hash, role (ENTREPRENEUR, STEWARD, ADMIN now; CUSTOMER, INVESTOR, SUPPLIER, PROGRAMME_OFFICER later), quota counters
sources, source_versions	source metadata, licence text, retrieval date, content hash, max_age_days, status
geo_units	id, parent_id, level, name, lgd_code, geometry (PostGIS), coverage_status
facilities	OSM-derived POIs: geometry, category, attractiveness proxy, completeness flag
quantities (the ledger)	all 13 fields from Part 4; parent_ids UUID[] with GIN index; kind, evidence_floor, resolution, uncertainty JSONB
policy_documents, policy_rules	document URL, sha256, retrieved_at; rule AST JSON, scheme_id, version, effective_from/to, status (DRAFT, APPROVED, RETIRED), approved_by
business_profiles	owner_id, sector template, user facts (user-owned)
scenarios, simulation_runs	scenario JSON, seed, model_version, git_hash, data_version_ids, outputs references, status
agent_runs, agent_run_steps	objective JSON, state, budgets used; step records with memento JSONB after every step
run_events	run_id, seq, event type, payload (redacted), timestamp; feeds SSE and replay
jobs	job type, payload, status, attempts, locked_by, run_at
approvals	run_id, action, requested_by, decided_by, decision
tasks (follow-ups)	run_id, text, linked_gap quantity, status
audit_events	append-only: actor, action, target, timestamp, hash of previous row optional

9.2 SQL for the central tables (reference)

```
CREATE TABLE quantities (
  id UUID PRIMARY KEY,
  definition_key TEXT NOT NULL,
  value DOUBLE PRECISION, unit TEXT NOT NULL,
  kind TEXT NOT NULL CHECK (kind IN ('OBSERVED','POLICY_RULE','USER_ASSUMED',
        'ESTIMATED','SCENARIO','CALCULATED','SIMULATED','COMMUNITY_REPORTED')),
  evidence_floor SMALLINT NOT NULL CHECK (evidence_floor BETWEEN 1 AND 4),
  geo_unit_id TEXT NOT NULL, resolution SMALLINT NOT NULL CHECK (resolution BETWEEN 0 AND 5),
  as_of DATE, period_start DATE, period_end DATE,
  uncertainty JSONB NOT NULL, source_refs JSONB NOT NULL DEFAULT '[]',
  method_id TEXT NOT NULL, model_version TEXT NOT NULL,
  parent_ids UUID[] NOT NULL DEFAULT '{}',
  quality_flags TEXT[] NOT NULL DEFAULT '{}', license_meta TEXT,
  owner_id UUID NULL,                       -- NULL = public; set = user-owned
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX quantities_parents_gin ON quantities USING GIN (parent_ids);

CREATE TABLE agent_runs (
  id UUID PRIMARY KEY, owner_id UUID NOT NULL, objective JSONB NOT NULL,
  state TEXT NOT NULL, steps_used INT NOT NULL DEFAULT 0, replans_used INT NOT NULL DEFAULT 0,
  model_calls_used INT NOT NULL DEFAULT 0, cost_usd NUMERIC(10,4) NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(), updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE agent_run_steps (
  run_id UUID REFERENCES agent_runs(id), seq INT, tool TEXT, input JSONB, output JSONB,
  memento JSONB NOT NULL, status TEXT NOT NULL, started_at TIMESTAMPTZ, ended_at TIMESTAMPTZ,
  PRIMARY KEY (run_id, seq)
);
CREATE TABLE run_events (
  run_id UUID, seq BIGINT, type TEXT NOT NULL, payload JSONB NOT NULL,
  at TIMESTAMPTZ NOT NULL DEFAULT now(), PRIMARY KEY (run_id, seq)
);
CREATE TABLE jobs (
  id BIGSERIAL PRIMARY KEY, kind TEXT NOT NULL, payload JSONB NOT NULL,
  status TEXT NOT NULL DEFAULT 'queued', attempts INT NOT NULL DEFAULT 0,
  locked_by TEXT, run_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
);
-- worker: SELECT ... FROM jobs WHERE status='queued' AND run_at<=now()
--         ORDER BY id FOR UPDATE SKIP LOCKED LIMIT 1;
```

Quantities are immutable: never UPDATE value or kind. Corrections create a new quantity and mark the old one superseded in quality_flags.

Part 10. API and user experience

10.1 Main API endpoints

Endpoint	Purpose	Roles
POST /auth/login, /auth/logout	Session cookie	all
POST /runs	Create an agent run from an objective; enqueue job	Entrepreneur
GET /runs/{id}	Run status, plan, step summaries	Owner, Admin
GET /runs/{id}/events	SSE stream of run events (polling fallback endpoint also provided)	Owner, Admin
POST /runs/{id}/answer	Provide an answer to a targeted question (AWAITING_USER)	Owner
POST /runs/{id}/approve	Approve or reject a gated action	Owner / Steward as configured
POST /scenarios, POST /scenarios/{id}/branch	Create or branch a scenario; run deterministic engines	Entrepreneur
GET /ledger/{quantity_id}	Quantity with ancestry graph for the Evidence Graph	Per-record access rules
GET /geo/{id}/indicators	Indicators with class, resolution, quality flags	all authenticated
GET/POST /policy/rules	List versions; steward creates draft and approves	Steward
GET /replay/{id}	Stored golden trace, labelled REPLAY	all
GET /admin/users, /admin/audit	Users, quotas, audit	Admin

10.2 Screens

Screen	What it shows (all from real state)	Key interactions
Economic Atlas	MapLibre map from PMTiles; layers: indicators (choropleth), OSM facilities, catchment polygons, demand cells, data-quality overlay (current, stale, estimated, unavailable)	Click any area or value to open the Provenance drawer (quantity, source, method, assumptions, resolution); filter layers; select sites
Scenario Laboratory	Editable parameters (price, costs, lambda, growth, loan terms); cash-flow fan chart; shortfall probability; tornado chart; site comparison table	Run, branch, compare side by side; job progress streamed from run events; time slider replaying monthly allocation on the map
Agent Console	Timeline of actual state transitions; each step expands to tool, redacted inputs and outputs, ledger records, verification results, replan reasons; budget meters	Approvals and questions inline; export trace; switch to replay
Policy Workspace	Source PDF with highlighted clause beside the rule tree coloured True/False/Unknown for the user's facts	Answer unknown-fact questions inline; see rupee impact overlay on cash flow
Evidence Graph	Cytoscape DAG: claim, quantities, calculations, sources	Click node for details; filter by kind or resolution
Steward workspace	Source versions, quality reports, draft rules, conflicts	Approve or reject rule versions; flag sources
Admin	Users, roles, quotas, audit log	Manage users and limits
Report view	Numbers as chips showing class and resolution; assumptions table; limitations	Export XLSX/PDF/JSON/CSV; hover chips for provenance

10.3 UX rules

- No animation unless it corresponds to a real event or a real model output. A progress bar must reflect real job progress.
- Every displayed number links to its quantity. Class and resolution are visible without clicking (badge or chip).
- The map must show gaps. Areas without usable data are shaded as such, not filled with estimates to look complete.
- Replay mode shows a persistent banner 'Replay of a recorded run' and cannot be mistaken for a live run.

- Role-specific navigation only shows allowed screens, but authorization is always enforced in the backend.

Part 11. Security, privacy and reliability

Area	Control (MVP unless noted)
Authentication	Password hashing (argon2 or bcrypt), JWT in httpOnly secure SameSite cookie, short expiry; seeded demo accounts for evaluators; OAuth in R1
Authorization	Role checks in service layer plus record ownership checks in repositories; Postgres row-level security as a second layer where practical. Frontend hiding is never relied on.
Input validation	Pydantic schemas on every request; size and type limits
Rate limiting and quotas	Per-IP and per-user limits; daily run quota; cost cap per run and per day
Secrets	Environment variables on the host; never committed; .env.example only
Prompt injection	Part 8.7: untrusted-text wrapping, typed outputs, plan validator, per-run tool allowlist
SSRF and fetch safety	Domain allowlist, block private/loopback/link-local ranges, timeouts, response size caps
Uploads (R1)	File type and magic-byte check, size cap, parse in a bounded worker, never execute content
Privacy	Shared or aggregate views only show cells with a minimum number of contributors (start with k = 5, tune). User business locations are snapped to a coarse grid in shared views. No personal financial data on shared maps.
Audit	Append-only audit_events for login, approvals, exports, rule activation, shared writes
Retention and deletion	Users can delete their runs and uploads; document retention periods in the privacy notice
Reliability	Idempotent job steps, checkpoint after every step, resume after worker kill, circuit breaker on providers, cached snapshot fallback for data sources
Output safety	Verifier gates; uncertainty labels always present; every report states it is decision support, not advice

Threats to test explicitly

- A policy PDF or community text containing instructions to the agent.
- A user attempting to read another user's run, quantity or upload (IDOR).
- Cost abuse by repeated runs; verify quotas and per-run caps stop it.
- A fetch tool pointed at an internal address.
- A frontend-only role bypass (call the API directly with the wrong role).

Part 12. Testing and evaluation

12.1 Test layers

Layer	Tools	Examples
Unit	pytest	EMI, break-even, Kleene tables, unit checks
Property-based	Hypothesis	Class never strengthens; allocation conserves demand; same seed gives identical results; EMI monotone
Golden numerical	pytest plus fixtures from an independent spreadsheet	Loan schedules, cash flows, catchments vs reference router
Contract	pytest parameterized	Every DemandAllocator and SimulationModel satisfies the same contract
Integration	Postgres test container	Ingestion, ledger persistence, job queue, resume after kill
Agent evaluation	benchmark harness	Section 12.2
Security	pytest plus manual	RBAC matrix, IDOR, injection documents, SSRF
End to end	Playwright in CI against deployed URL	Login, Atlas, drill-down, run scenario, live progress, scheme evaluation, report, resume, replay

12.2 Agent benchmark design

Build roughly 60-100 tasks across Loops A, B and C with gold labels. Include answerable tasks, unanswerable (missing data) tasks, contradictory-source tasks, village-level resolution traps, scheme edge cases and injected tool failures. **Split into development, validation and frozen test sets.** Do not tune prompts or gating rules on the frozen test set. Have someone other than the task author label at least a sample.

System	Description
B1	Single-pass LLM with tool access, no gating
B2	Fixed analytical pipeline (no adaptivity)
B3	Full agent with the verifier gates turned off (ablation)
B4	Full ArthLens agent with the ledger and gates

Metric	Mechanical definition
Unsupported numeric claim rate	Share of numbers in the output with no ledger id, or whose ledger class contradicts the label used
Class-mislabel rate	Share of claims described as a stronger class than the computed floor
Resolution-overclaim rate	Share of claims at a finer resolution than the evidence supports without a registered method
Correct insufficient-evidence rate	On unanswerable tasks, share where the system reports the gap instead of answering
Eligibility accuracy	Agreement with human-labelled truth table
Numerical correctness	Agreement with reference calculations within tolerance
Completion and recovery	Task completion rate; recovery success under injected failures
Cost and effort	Model calls, tool calls, latency and USD per completed task
Injection success	Number of tool authorizations triggered by document text (target: zero)

Report bootstrap confidence intervals; with small N avoid claims of significance on small differences. **Does agentic execution add value?** Compare B4 with B2 on tasks where the correct path depends on intermediate findings, and with B1 on cost and correctness. If B2 matches B4 at lower cost, report that honestly.

12.3 Acceptance criteria (set before running; initial hypotheses)

- Same scenario, seed, model version and data versions give identical outputs.
- Zero constraint violations in generated scenarios; zero free numbers pass the renderer.
- B4 unsupported-claim rate lower than B1 with non-overlapping confidence intervals. If not, document the result.
- No performance targets are claimed until measured: record p50/p95 API latency, job duration by type, query timings and cost per run on the deployed system.

Part 13. Build order, tasks and schedule

Task IDs are used in commit messages and Antigravity prompts. Do not start a phase until the previous phase's acceptance column is satisfied. 'Human' tasks must be done by a person, not delegated to the AI agent.

Phase 0: Research and validation (weeks 1-2)

ID	Task	Owner	Acceptance
P0-1	Verify each source in the register (Part 7); write docs/data-sources/*.md with licence and GO/NO-GO	A+B (human)	Every MVP source has a live-verified entry
P0-2	Fetch PMFME, PMEGP, PM-MUDRA official guidelines; hash; extract parameters by hand	B (human)	PDF hashes and parameter table with page references
P0-3	Competitor checks: ONDC, Udyam, MSME Sampark, two deep-research agents; name check for ArthLens	A	Competitor table updated with evidence
P0-4	Choose pilot district by the go/no-go gate (Part 7.3); choose two business templates and identify parameter sources	A+B	Written decision with evidence
P0-5	Write ledger invariants as tests (Part 4.7) and the benchmark task format	A	Failing tests committed
P0-6	Confirm hosting limits (Part 14) and LLM provider terms	B	docs/open-questions.md resolved or listed

Phase 1: Domain, ledger, auth (weeks 3-4)

ID	Task	Owner	Acceptance
P1-1	Monorepo, CI, Docker compose with Postgres+PostGIS	B	CI green on empty tests
P1-2	Domain classes, enums, interfaces from Part 5	A	Contract-test skeletons exist
P1-3	Quantity, ledger DAG, propagation, uncertainty, units	A	All Part 4.7 property tests pass
P1-4	Nine verification specifications and LedgerVerifier; ReportRenderer	A	Spec unit tests and free-number rejection pass
P1-5	Alembic migrations (Part 9), repositories, in-memory fakes	A	Integration tests on a Postgres container
P1-6	Auth, roles, RBAC middleware, audit events, rate limiting	B	RBAC matrix tests pass; IDOR test passes
P1-7	Initial UML (use-case, class, state machine)	A+B	Diagrams in docs/uml

Phase 2: Real-data foundation and Atlas (weeks 5-6)

ID	Task	Owner	Acceptance
P2-1	DatasetIngestionPipeline plus adapters for verified sources	A	Each adapter ingests fixture and live data
P2-2	Geographic hierarchy, LGD crosswalk, boundaries to PMTiles	A	Navigation from nation to district works; second-district config test
P2-3	Data-quality layer and staleness flags	A	Atlas shades quality states correctly
P2-4	Atlas UI with MapLibre; Provenance drawer; Evidence Graph	B	Every displayed number opens its ledger record

Phase 3: Computational engine (weeks 7-9)

ID	Task	Owner	Acceptance
P3-1	Road subgraph extractor, Dijkstra, catchments	A	Matches NetworkX; reference router comparison documented
P3-2	Nearest and Huff allocators; popweight_v1 downscaling	A	Contract tests; conservation tests
P3-3	Finance engine, loan engine with moratorium/subsidy parameters	A	Golden spreadsheet tests
P3-4	Vectorized Monte Carlo, OAT sensitivity; reproducible runs	A	Seed reproducibility; convergence reporting

ID	Task	Owner	Acceptance
P3-5	Rule engine: nodes, Kleene, visitors, versioning; encode three schemes from verified parameters	B	Truth-table tests; steward sign-off
P3-6	Scenario Laboratory UI; Policy Workspace UI	B	Run, branch, compare against real outputs

Phase 4: Agent orchestration (weeks 10-11)

ID	Task	Owner	Acceptance
P4-1	Tool base class, 13 tools, decorators, registry	B	Tool contract tests; decorators order tested
P4-2	RunState State pattern, budgets, memento persistence, job worker	B	Illegal transitions raise; resume after worker kill
P4-3	ILLMClient, router, circuit breaker, replay and fake clients, firewall	B	Full run with FakeLLMClient; injection tests pass
P4-4	Goal interpreter, planner, plan validator, recovery rules	B	Decision-table tests (Part 8.4)
P4-5	SSE events, Agent Console UI	B	Console shows real events in order

Phase 5: End-to-end workflows (weeks 12-13)

ID	Task	Owner	Acceptance
P5-1	Loop A, B, C complete flows	A+B	Three scripted demos pass end to end
P5-2	Report builder, XLSX/PDF/JSON/CSV exporters; XLSX recalculation equals app	B	Export tests
P5-3	Follow-up task generation from verifier gaps	A	Tasks created from unresolved gaps
P5-4	Record three golden traces for replay mode	B	Replay labelled and loads offline

Phase 6: Evaluation (week 14)

ID	Task	Owner	Acceptance
P6-1	Author benchmark, freeze test set, independent label sample	A+B	Frozen set committed with hash
P6-2	Run B1-B4; compute metrics with confidence intervals	A	Results table published, whatever it shows
P6-3	Measure latency, job duration, cost per run	B	Metrics recorded on deployed system
P6-4	Security tests and fixes	B	Threat tests pass

Phase 7: Deployment (weeks 15-16)

ID	Task	Owner	Acceptance
P7-1	Deploy frontend, API, worker, database, storage	B	Public URL reachable from a fresh browser
P7-2	Migrations, demo data, demo accounts, quotas, secrets	B	Fresh deploy reproducible from README
P7-3	Playwright smoke suite in CI against the public URL	B	Green after each release
P7-4	Final docs: README, UML, API, limitations, honest results	A+B	Handbook Definition of Done met

13.2 Cut order if time runs short

1. Sobol sensitivity (already R1)
2. Time-slider playback
3. Hindi
4. Second business template
5. Policy version diff
6. Hosting extras (error tracking)

Never cut: the ledger and verifier, the finance and rule engines, the agent state machine with resume, the benchmark, and the public deployment with replay mode.

13.3 Minimum Demo Slice (if only about six weeks are available)

- Pilot district, one business template (retail), one scheme (the one with the most readable guidelines).
- Ledger with V1, V2, V4, V6 and the free-number renderer; Quantity propagation tests.
- Finance engine (break-even, EMI, cash flow), OAT tornado, Monte Carlo; Nearest allocator and Huff.
- Rule engine for one scheme; Loop C only plus a simplified Loop B for 2 sites.
- Agent with state machine, budgets, resume, FakeLLM and one real provider; Agent Console.
- Replay mode with one recorded run; deploy. State plainly that this is a slice of the plan.

Part 14. Deployment and cost

[VERIFY] everything in this part against each vendor's current pricing and documentation page before committing. Free tiers change, sleep, expire or restrict use. The research below came from third-party summaries and must be confirmed.

14.1 What research suggested

- Render free web services spin down after about 15 minutes idle and take roughly a minute to wake; the free Render Postgres expires after about 30 days; the Starter instance is about 7 USD per month and avoids spin-down.
- Neon's free plan offered about 0.5 GB storage per project with scale-to-zero and limited compute hours.
- Vercel Hobby is for personal, non-commercial use: fine for coursework, not for a commercial startup.
- The earlier draft proposed Supabase (Postgres + PostGIS), Cloudflare R2 (PMTiles), Hugging Face Spaces (container for API and worker) and Vercel (frontend). Each is plausible but unverified here: confirm PostGIS support, free storage and egress limits, whether a Space can run a long-lived worker alongside the API, sleep behaviour, and acceptable-use terms.

14.2 Recommended architecture [DECISION, with VERIFY items above]

Layer	Choice	Fallback
Frontend	Vercel Hobby (or Cloudflare Pages)	Any static/Node host
API	Container on an always-on instance (for example a paid starter instance) to avoid cold starts during evaluation	Free instance with a warm-up ping and a loading screen
Worker	Second process or service from the same image	Run the worker loop inside the API container if the host allows
Database	Managed Postgres with PostGIS (verify extension and size limit against the pilot data)	Different managed provider
Object storage	S3-compatible bucket for PMTiles, PDFs, exports	Static hosting for PMTiles
LLM	Provider behind ILLMClient with router, circuit breaker and replay	ReplayLLMClient and replay mode

14.3 Cost model

Monthly cost = API instance + worker instance + database (if paid) + storage + LLM spend. LLM spend = runs x model calls per run x tokens per call x price per token. No factor is measured yet, so no estimate is given here. Set a hard daily and monthly cap in the budget governor, measure cost per run in Phase 6, and compute the real figure then.

14.4 Demo reliability

- Pre-record three golden traces (one per loop). Replay mode serves them from stored events and shows the REPLAY banner.
- Cached snapshots of every data source in object storage so ingestion never depends on a live government site during a demo.
- Demo accounts per role with daily run quotas; when the cap or provider fails, the UI offers replay.

14.5 Deployment and smoke procedure

1. CI runs unit, property, contract and integration tests, then builds the image.
2. Deploy the image; run Alembic migrations; seed demo data and demo accounts.
3. Run the Playwright smoke suite against the public URL: login for each role, load Atlas, drill into one number, run a short Monte Carlo job and watch events stream, evaluate a scheme, generate and download a report, kill the worker mid-run and confirm resume, open replay mode.
4. Verify from a fresh browser session on a different network.

Part 15. Definition of done and honest claims

15.1 Definition of done (all must be true)

- Public URL works from a fresh browser with no installation; demo accounts documented.
- The three MVP workflows run end to end and are reproducible from stored scenario, seed and versions.
- Ledger invariant tests, finance golden tests, rule-engine truth tables and contract tests all pass in CI.
- Every number in every report is a ledger chip with class and resolution; the renderer rejects free numbers.
- The agent run can be killed and resumed; budget exhaustion returns a partial result with gaps listed.
- Backend RBAC and IDOR tests pass; injection documents authorize zero tools.
- Benchmark B1-B4 results are published exactly as measured, with the frozen test set untouched during tuning.
- UML diagrams match the code; docs list limitations and all remaining [VERIFY] items resolved or explicitly listed.

15.2 What may be claimed (and what may not)

May claim after the evidence exists	Must not claim
Measured reduction in unsupported claims vs a stated baseline on the stated benchmark	That the ledger idea is novel, before a literature check
Bit-identical reruns verified by N tests	Predictive accuracy of demand or revenue without observed outcome data
Resume after injected failure in X of Y runs	Performance, scale or cost figures that were not measured
Correct eligibility on a labelled set of N cases	Guaranteed loan approval or business success
The system degrades gracefully on a second district via configuration	Village-level precision from district-level data

15.3 Draft resume description (fill numbers only after measuring)

ArthLens: Agentic Economic Decision-Support Platform. Built a provenance-aware engine linking public data, versioned policy rules, geospatial analysis and reproducible financial simulation. Implemented a stateful agent orchestrator with typed tools, bounded replanning, checkpoint/resume and a verifier that blocks unsupported numeric claims ([X%] to [Y%] versus a single-pass baseline on a [N]-task benchmark).

Part 16. Corrections to earlier drafts

The earlier vision and master-specification drafts contained statements that must **not** be copied into code, UI text, reports or the viva without verification. Use this list as a checklist.

Earlier statement	Problem	Use instead
'Cryptographic provenance ledger'	No cryptography is required or planned.	'Immutable, versioned provenance ledger'. Hash-chained audit rows are an optional extra.
'Mathematically blocks' district data shown as village data	It is blocked by deterministic rule checks, not by mathematics.	'Rule-based verification blocks...'
Evaluating 2,000 trajectories 'in under 30 milliseconds'	Not measured.	Report measured latency from the deployed system.
'78% of profit variance' and similar figures in examples	Example outputs, not results.	Mark [ILLUSTRATIVE]; real figures come from computed ledger quantities.
Specific scheme parameters (subsidy percentage, caps, contribution percentage, loan limits)	Not verified against current guidelines in this work.	Read official PDFs in P0-2 and encode only those values, with page references.
Specific LLM model names and free-tier request/token numbers; a model named 'Gemini 3.8'	Unverified and quickly outdated.	Configurable providers; confirm at build time.
'Hugging Face Spaces free 16 GB RAM', Supabase PostGIS, R2 free tier	Unverified here.	Verify in P0-6.
Counts such as 760+ districts and 6,000+ subdistricts	Unverified; depend on the boundary source and date.	Derive counts from the chosen verified source.
'All 43 capabilities and 7 roles end-to-end' via six engines	Not achievable in one semester.	Tiering in Part 2; 3 roles and the MVP capabilities.
42-day plan	Unrealistic for the full scope.	16-week plan or the Minimum Demo Slice.
Sobol, MILP, six export formats, Hindi in MVP	Scope creep.	R1 / Advanced as tiered.
Adapters named for Census LGD, Udyam, RBI (as if known to work)	Availability of machine-readable access not confirmed.	Verify in Phase 0; drop any that fail.
Recharts and React Flow in one place, ECharts and Cytoscape in another	Inconsistent.	ECharts and Cytoscape.js only.
'Interactive diagram below' at the end of the master spec	No diagram existed in the document.	Create diagrams under docs/uml.
'Pan-India' product claim	Analysis is limited to analysable areas.	Pan-India architecture and navigation; analysis where evidence exists.

Appendix A. Ready-to-use prompts for Antigravity

Paste A.0 at the start of every session, then the prompt for the current task. Review each result against the acceptance column in Part 13 before moving on. Prompts are starting points: adapt wording, never the rules.

A.0 Master prompt (paste first, in every new Antigravity session)

You are building ArthLens from the ArthLens Build Handbook (v1.0). Read the handbook sections the task refers to before coding.
Rules: (1) Never invent data, scheme parameters, URLs, limits or results. If something is marked [VERIFY], stop and add it to docs/open-questions.md.
(2) The LLM never computes numbers; all maths/eligibility is deterministic Python. (3) Implement to the interfaces in Part 5; do not change signatures without updating docs/uml and contract tests in the same commit.
(4) Write the tests named in the handbook FIRST, then the implementation. (5) Every external dependency goes behind an adapter with a fixture/fake. (6) Work on ONE task ID at a time; make small commits prefixed with the task ID.
(7) At the end of each task: run the full test suite, list what is done, what is not, and any deviations. Add deviations to docs/DECISIONS.md. (8) Do not mark a [VERIFY] item as resolved yourself; a human does that.

A.1 Phase 1 tasks P1-3 and P1-4: ledger and verifier

Task P1-3/P1-4. Implement backend/arthlens/ledger per handbook Part 4. Steps: (a) write Hypothesis property tests from Part 4.7 and unit tests for the nine specifications V1-V9; (b) implement the immutable Quantity dataclass, Kind enum, evidence_floor combination rule (4.2), resolution rule with a DownscalingRegistry (4.3), uncertainty combination (4.4) with interval arithmetic, Pint unit checks; (c) implement the DAG ledger with cycle rejection and ancestry queries; (d) implement each specification as its own class and LedgerVerifier as an AndSpecification chain returning per-spec results; (e) implement ReportRenderer that accepts only {{q:UUID}} placeholders and rejects free numeric literals. Done when all tests pass and coverage for ledger/ is reported.

A.2 Phase 1 task P1-5/P1-6: database and auth

Task P1-5/P1-6. Create Alembic migrations for the tables in handbook Part 9 (use the SQL as the reference), repository interfaces plus Postgres and in-memory implementations, and FastAPI auth: password hashing, JWT in httpOnly cookie, roles table, RBAC dependency that checks role AND record ownership in repositories, rate limiting, append-only audit events. Write an RBAC matrix test calling the API directly with each role against each endpoint, and an IDOR test where user A requests user B's run. Done when those tests pass in CI against a Postgres+PostGIS container.

A.3 Phase 2: ingestion and Atlas

Task P2-1..P2-4. Only use data sources whose docs/data-sources/<id>.md has a GO line. Implement DatasetIngestionPipeline (template method, 10 steps in handbook 7.5) and one adapter per GO source, each with a fixture-based test and a live test marked optional. Reconcile geography to LGD codes via a crosswalk table; store OBSERVED quantities with as_of, licence_meta and quality flags (STALE if beyond max_age). Build boundary PMTiles and the Atlas page (MapLibre) with a data-quality overlay and a Provenance drawer showing a quantity's ancestry. Add a test proving a second district can be added by configuration only. Do not fabricate any dataset; if a field is missing, expose it as UNAVAILABLE.

A.4 Phase 3: geospatial, demand, finance

Task P3-1..P3-4. Implement our own Dijkstra (heapq) for catchments with tests against NetworkX on random graphs; DemandAllocator interface with NearestFacilityAllocator and HuffAllocator plus shared contract tests (conservation, probabilities sum to 1, large-lambda limit); popweight_v1 downscaling registered in the DownscalingRegistry with a conservation test; finance engine as a Template Method with golden tests against fixtures from an independent spreadsheet; loan engine with moratorium and subsidy mechanism as PARAMETERS (do not assume values); vectorized Monte Carlo with PCG64 seed, convergence reporting, and OAT sensitivity. Outputs must be registered in the ledger with correct kind, floor, resolution and parents. Do not state any run-time figures.

A.5 Phase 3: policy rule engine

Task P3-5. Implement RuleNode (And, Or, Not, LeafPredicate) as a Composite with Kleene three-valued logic, and Visitors: EligibilityEvaluator (returns status, unknown facts, trace), PolicyDiff, FinancialImpactExtractor. Rules are loaded from versioned JSON with effective dates and an as-of argument. Write exhaustive truth-table tests first. Create the JSON files for the three schemes ONLY from parameters that a human has recorded in docs/data-sources/schemes.md with page references; if the file is missing or incomplete, stop and ask.

A.6 Phase 4: agent engine

Task P4-1..P4-5. Implement Tool (template method) and the tools in handbook 8.5; decorators in the order Budget, TimeoutRetry, ProvenanceRegistration, AuditLogging; AgentRun with State-pattern RunState subclasses covering every state and transition in 8.2 (illegal transitions raise); ExecutionMemento persisted after each step; a Postgres job worker using FOR UPDATE SKIP LOCKED; ILLMClient with router, circuit breaker, ReplayLLMClient and FakeLLMClient; untrusted-text firewall; PlanValidator; recovery decision table (8.4); SSE endpoint reading run_events. Tests: a full Loop B run with FakeLLMClient and in-memory repositories; kill-and-resume; budget exhaustion returns a partial result; injection documents authorize zero tools.

A.7 Phase 5-7: workflows, benchmark, deployment

Task P5-1..P7-4. Wire Loops A, B, C end to end using only registered tools. Implement the report builder (ledger chips only) and XLSX/PDF/JSON/CSV exporters; XLSX formulas must recalculate to the app's values (test). Build the benchmark harness for systems B1-B4 with dev/validation/frozen splits and the metrics in handbook 12.2; do not tune on the frozen set. Create deployment files, README with exact steps, env.example, Playwright smoke tests, and the replay mode with banner. Report measured results exactly; leave metrics blank if not measured.

Appendix B. Sources consulted during research

Third-party and secondary sources are marked (secondary). Confirm anything important against the official source.

Topic	Source / URL
data.gov.in API usage	pypi.org/project/datagovindia (community wrapper, secondary); publicapi.dev/open-government-india-api (secondary); informaticsweb.nic.in/sites/default/files/ogd.pdf
SHRUG	theigc.org (SHRUG publication page); esoc.princeton.edu SHRUG resource page; openknowledge.worldbank.org SHRUG documentation
RBI deposit and credit statistics	website.rbi.org.in press releases on quarterly statistics of scheduled commercial banks; dbieold.rbi.org.in DBIE brochure 2022
myScheme	myscheme.gov.in/about; negd.gov.in/myscheme
EconAgent	arxiv.org/abs/2310.10436; aclanthology.org/2024.acl-long.829
LangGraph durable execution	docs.langchain.com/oss/python/langgraph/durable-execution
Hosting free tiers (secondary)	render.com/articles/platforms-with-a-real-free-tier-for-developers-in-2026; livemy.app/blog/render-pricing; agentdeals.dev comparison pages
GeoIQ	app.dealroom.co/companies/geoiq (status listing, secondary)
PhonePe PulsePro	phonepe.com/press (PulsePro launch); yourstory.com 2026/07 article
GapMaps Live India	gapmaps.com
Retail Vista (Jio)	glass.jiocommerce.io/agents/retail-vista

Open questions to close in Phase 0

- Which exact datasets and resource ids will be used, with licences?
- Which pilot district passes the go/no-go gate?
- Which three schemes are current, and what are their parameters from the official PDFs?
- Boundary source and official-map requirements for depicting India.
- Hosting choice after verifying free-tier and PostGIS limits.
- Literature check for the Epistemic Ledger novelty claim.
- Name availability for ArthLens.